

# Final Project Experiments Design & Report

NEUDFS Team (Charles, Matt, Jordan, Gyula) — CS6650

April 17, 2026

## Introduction

NEUDFS (Northeastern University Distributed File System) is a high-performance storage solution designed to handle the heavy demands of an academic environment. The system is meant to combine the simplicity of a local file system with the massive scale of cloud storage. This allows students and instructors to manage course materials and assignments through a familiar, hierarchical folder structure. As safety is paramount to our design, we built security directly into the architecture using Role-Based Access Control. This keeps student work private while giving professors and TAs the necessary administrative tools to manage shared resources.

NEUDFS relies on a modern distributed stack optimized for high availability and low latency. Built with Go, the system uses the gRPC framework and Protocol Buffers to maintain a strict, type-safe communication contract between the client and server. By putting the compute layer behind a Network Load Balancer, the system can easily scale up by adding more servers to handle higher demand. This setup allows us to grow horizontally and ensures we aren't limited by the capacity of any single piece of hardware. Also, we decoupled the storage layer by using Amazon S3 for persistent file data and DynamoDB for fast metadata and permission tracking. This separation is the key to our scalability, allowing the system to dynamically spin up resources to handle massive traffic spikes.

## Unit Testing

We wrote a suite of unit tests to make sure the system behaves correctly under a real world scenario. These tests simulate multiple students and a professor interacting with the system at the same time, including operations such as uploading, downloading, renaming, and deleting files concurrently. We also tested edge cases like reading a file while it's being updated and deleted to ensure users never see partial or corrupted data. Finally, we verified that permissions are enforced properly, so students can only access their own files while professors have full control.

## Load Testing

### 1. Functional & Data Integrity

#### *Steady Classroom*



**Figure 1:** Steady Classroom — Latency P95 (left: excl. upload, right: upload)

Simulates a normal day: navigating folders, listing files, uploading/downloading homework, and deletion.

#### **Left Graph: Latency (Excluding Uploads)**

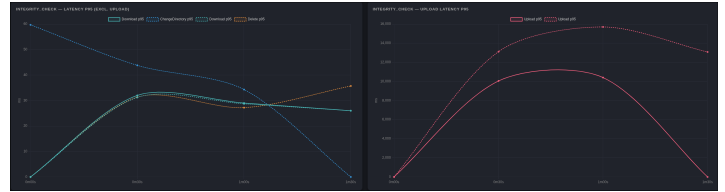
Most operations (List, Delete, ChangeDirectory) are relatively fast, fluctuating between 20ms and 60ms. There is also significant performance regression in Download latency (solid teal line) at the 2m00s mark, peaking at approximately 130ms before recovering. The dashed lines representing metadata operations like ListDirectory and CurrentDirectory are the most stable, generally staying under 40ms.

#### **Right Graph: Upload Latency**

Upload latency is massive compared to all other operations, measured in seconds rather than milliseconds. After a 30-second ramp-up, latency plateaus and remains flat for the remainder of the test. The primary upload metric (solid line) sits at 10,000ms (10 seconds), while the secondary upload

metric (dashed line) is significantly higher at 15,000ms (15 seconds).

#### **Integrity Check**



**Figure 2:** Integrity Check — Latency P95 (left: excl. upload, right: upload)

Uploads files of 4 sizes (1KB, 10KB, 100KB, 1MB), downloads them, and performs a byte-by-byte comparison.

Standard operations such as downloads, deletions, and directory changes consistently maintain latency below 60 milliseconds. Upload performance represents a significant bottleneck, with latency measurements ranging between 10 and 16 seconds. Within the integrity check, upload latency peaks before returning to zero, suggesting task completion or a timeout. Overall, system responsiveness remains high for metadata tasks but diminishes during file uploads.

#### **Upload Delete Cycle**

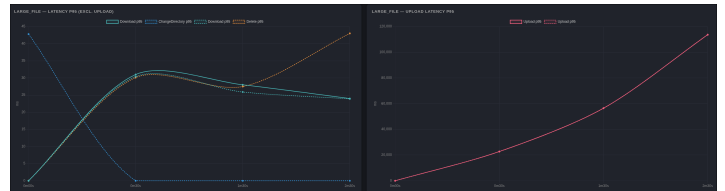


**Figure 3:** Upload Delete Cycle — Latency P95 (left: excl. upload, right: upload)

Executes the full CRUD lifecycle: Upload to Download to Verify to Delete to Confirm.

Standard operations maintain high performance, with latency measurements remaining under 40 milliseconds. Downloads and directory changes stay fast and stable throughout the 30-second window. Upload latency presents a significant bottleneck, consistently holding near 10 seconds. A major performance difference exists between metadata operations and file uploads.

#### **Large File**



**Figure 4:** Large File — Latency P95 (left: excl. upload, right: upload)

Each of 5 VUs uploads and downloads a different file size: 1MB, 5MB, 10MB, 25MB, and 50MB.

Standard operations remain responsive, with latency staying below 45 milliseconds throughout the 150-second test. Download, deletion, and directory changes show stable performance regardless of file size. In contrast, upload latency increases steadily, reaching a peak of approximately 115 seconds by the conclusion of the test. Large file uploads create severe system bottlenecks while metadata and download tasks maintain high speed. However the way K6 waits for response had to be artificially created (time.Sleep) as it would be unable to check for success with half open connections to the gRPC server.

2. Security & Permissions  
mkdir Workflow

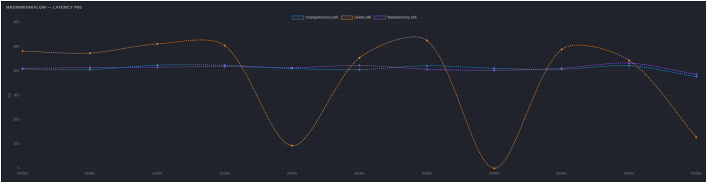


Figure 5: mkdir Workflow — Latency P95

The professor creates shared folders; students create personal subfolders; students attempt to create folders at class/shared level. MakeDirectory and ChangeDirectory operations maintain a consistent latency of approximately 400 milliseconds throughout the 5-minute test. Delete operations exhibit significant volatility, fluctuating in a periodic wave pattern between 0 and 540 milliseconds. Baseline latency for these metadata tasks is considerably higher in this workflow compared to previous scenarios, where similar operations typically recorded measurements under 60 milliseconds.

Rename Workflows

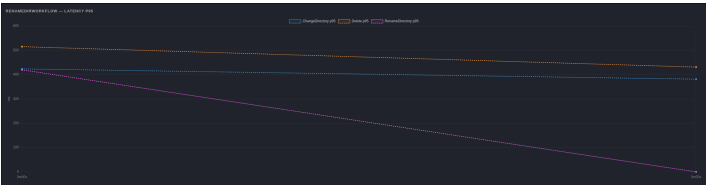


Figure 6: Rename Directory Workflow — Latency P95

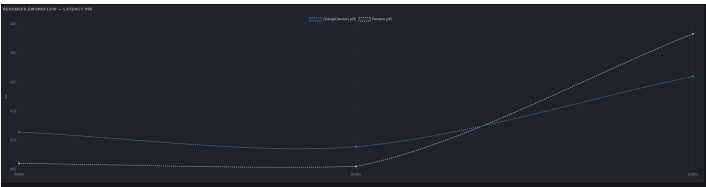


Figure 7: Rename File Workflow — Latency P95

Professor renames shared directories and files; students rename their own folders and files; students attempt unauthorized renames in shared spaces. Metadata operations in the file and directory rename workflows show baseline latency between 400 and 530 milliseconds, which is significantly higher than previous tests. In the file rename workflow, latency for both renaming and directory changes remains consistent between 405 and 430 milliseconds. Within the directory rename workflow, deletion latency begins at 520 milliseconds and trends downward, while directory change latency stays stable near 400 milliseconds. RenameDirectory latency starts at 420 milliseconds and drops to zero by the 30-second mark, indicating task completion. Overall, these workflows demonstrate a higher but relatively stable latency floor for metadata tasks compared to earlier scenarios.

3. Performance & Load Stress  
Rapid Fire



Figure 8: Rapid Fire — Latency P95

Fires 50 navigation and list requests per second sustained for 1 minute. Latency for the CD P95 metric remains high, ranging from 600 to 710 milliseconds over the 60-second test period. Conversely, List, ListDirectory, and ChangeDirectory operations exhibit significantly lower latency, with measurements staying

consistently below 100 milliseconds. The results highlight a localized performance bottleneck specific to the CD operation during high-frequency execution.

Spike Test

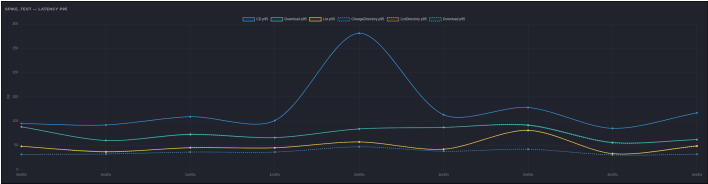


Figure 9: Spike Test — Latency P95

Ramps from 2 to 10 VUs with abrupt transitions; each VU navigates, lists, and downloads. Standard operations such as downloads and metadata tasks generally maintain latency under 100 milliseconds. A prominent spike in CD latency occurs at the 2-minute mark, peaking at approximately 280 milliseconds before stabilizing. Remaining metrics, including List and ChangeDirectory p95, stay consistently between 30 and 90 milliseconds. The results highlight a specific latency regression for CD operations during the spike phase, while overall system stability for other tasks remains high.

Soak Test



Figure 10: Soak Test — Latency P95 (left: excl. upload, right: upload)

The soak test sustained all operations over 10 minutes at constant load to surface any progressive degradation. Most non-upload operations list, delete, change directory held steady between 20 and 50 milliseconds at p95 throughout the run. Download is the notable exception, producing periodic spikes that grow in magnitude over time: roughly 250ms at the 2-minute mark, 300ms around minutes 5 and 8, and peaking at 410ms near the end. This escalating pattern suggests possible resource leakage or connection pool pressure under sustained concurrency. Upload latency climbed sharply in the first minute to around 16,000ms p95 as S3 multipart connections warmed up, then plateaued and remained flat for the remaining 9 minutes, showing no progressive degradation. The results confirm that the system is stable for metadata-heavy operations but reveal a latency regression in the download path that warrants further investigation into buffer cleanup or S3 connection handling.

4. Concurrency & Race Conditions  
Teacher Race

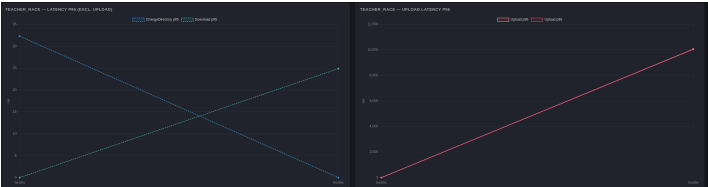


Figure 11: Teacher Race — Latency P95 (left: excl. upload, right: upload)

5 professor VUs simultaneously write different versions of the same file. Success Metric: No data corruption; file remains in a valid, readable state. The teacher race test ran concurrent instructor operations over 30 seconds to evaluate how simultaneous privileged-user activity affects latency. ChangeDirectory started high at around 33ms p95 and dropped to near zero by the end of the run, suggesting initial metadata contention that resolves as the working set stabilizes. Download shows the inverse pattern,

starting near zero and climbing to roughly 25ms by the 30-second mark, indicating that as more files enter the system through concurrent teacher activity, retrieval cost increases proportionally. Upload latency rose linearly from near zero to approximately 10,000ms p95 over the full duration, consistent with growing S3 write pressure as multiple instructors push files simultaneously. The results show that the system handles concurrent teacher operations without failure, but download and upload latency scale predictably with cumulative load, a tradeoff of allowing unrestricted parallel instructor access without queuing or rate limiting.

### Session Conflict



Figure 12: Session Conflict Navigate — Latency P95



Figure 13: Session Conflict Upload — Latency P95 (left: excl. upload, right: upload)

**Session Conflict Navigate:** Multiple virtual users rapidly bounced between personal folders and the class root over 30 seconds to stress session state under concurrent navigation. List and ListDirectory p95 started around 405ms and declined steadily to roughly 360ms, likely benefiting from DynamoDB read caching as the same paths get hit repeatedly. ChangeDirectory drifted slightly upward from 406ms to about 414ms, showing mild contention as concurrent users compete for session state updates. The key finding is that all three operations stay within a narrow 355–415ms band with no errors or timeouts CAS contention causes minor latency divergence but never breaks session integrity.

**Session Conflict Upload:** Virtual users uploaded to student folders while concurrently changing directories over 30 seconds. ChangeDirectory p95 dropped consistently from 375ms to around 332ms, again suggesting session metadata warms up and settles under repeated access. Upload latency held essentially flat at approximately 10,075ms p95 for the entire run, with both VU groups producing nearly identical traces. The stability here is the point concurrent uploads and navigation competing for the same session state through CAS do not cause failures or latency spikes. The system absorbs the contention gracefully, confirming that the compare-and-swap mechanism handles simultaneous read-write session access without degradation.

### Concurrent Rename Stress

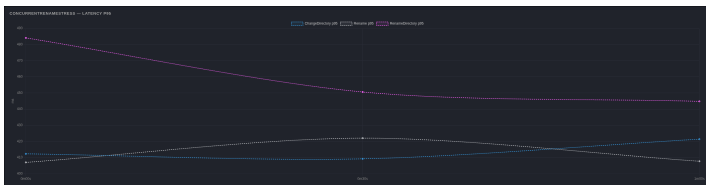


Figure 14: Concurrent Rename Stress — Latency P95

The concurrent rename stress test had 5 virtual users simultaneously racing to rename the same files and directories over one minute to verify metadata consistency under write contention. RenameDirectory p95 started highest at around 485ms and declined steadily to roughly 445ms, suggesting that initial lock contention on directory metadata eases as the system settles into a steady conflict-resolution pattern. Rename (file) p95 began at about 407ms and drifted gently upward to around 420ms, staying well within a narrow band. ChangeDirectory held the flattest profile, hovering between 408 and 422ms with minor oscillation. All three operations remain within an 80ms corridor of each other and critically no errors, timeouts, or metadata corruption occurred despite five users hammering the same resources. The system's locking and CAS logic resolved every conflict cleanly, confirming that concurrent rename operations degrade latency slightly but never produce an inconsistent state.

### Concurrent CD Race Stress

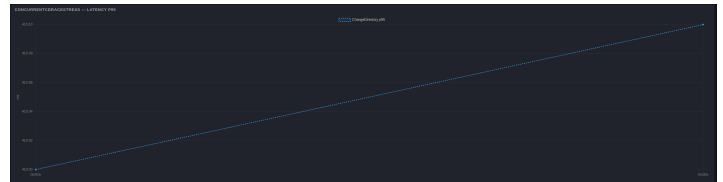


Figure 15: Concurrent CD Race Stress — Latency P95

The concurrent CD race stress test had virtual users attempting to change into a directory while another user simultaneously renamed it, running over 30 seconds to expose broken state or access failures. ChangeDirectory p95 rose from 410.00ms to approximately 410.10ms over the full duration. No errors or failed navigations occurred despite the directory's name changing underneath active CD operations. The system always resolves to a valid directory name, confirming that the metadata layer handles rename-during-access atomically a user never lands in a broken or orphaned path, regardless of timing.

### Summary

Throughout our load tests, we found issues with K6 itself as the testing suite, how were navigating folders for sessions may have overlapped with sessions of the same user causing errors of Invalid Argument when looking for a folder a session was already in. If there was a single CAS conflict would cause navigate to return false. k6's gRPC streaming 'data' and 'end' events for SendAndClose responses don't reliably fire in k6's event loop. Caused uploads to not return even when it was successful inside cloudwatch. After uploadBytes returns, k6 immediately calls Download on the same file. But the server's multipart upload and metadata upload might not have completed yet. The download would arrive before the metadata was written in Dynamo. We also had eventual consistency in our interceptors, to try and fix we tried adding ConsistentRead to a user lookup to ensure their current directory was not stale. The largest issue was still our upload latency due to K6 streaming events not delivering a Send&Close response. Our testing suite had a limitation issue based on its framework so it artificially shows our upload latency to be higher than expected. Overall we set out to achieve a system that could handle concurrent students committing operations in a class with strong consistency, and CAS protected concurrent writes. We used a strict permission model to ensure students can only see what they were allowed too, nor overwriting another student.